

SuperDFS Heurística

Exploración en Profundidad Guiada por Pila e Ignorados

Dario Turco

Funcionamiento de la Heurística

Premisa

La instancia es **resoluble** (existe un director). El estado actual s **pertenece al director**.

DFS guiado por dos estructuras auxiliares:

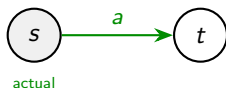
Estructuras:

- **Pila (STACK)** – no-controlables diferidas
- **Ignorados (IGNORE)** – controlables no elegidas

En cada estado s :

- 1 Si hay no-controlables desde s :
elegir la primera, apilar el resto
- 2 Si solo hay controlables desde s :
picker de familia, resto $\rightarrow$ ignorados
- 3 Sin transiciones desde s :
pop **STACK** $\rightarrow$ pop **IGNORE**

Caso 1 – Única transición controlable



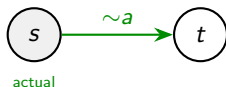
Desde s : solo una transición controlable a .

► Elegir $a \rightarrow t$

- STACK: sin cambios
- IGNORE: sin cambios
- Siguiente estado: t

No hay decisión que tomar; única opción válida.

Caso 2 – Única transición no-controlable

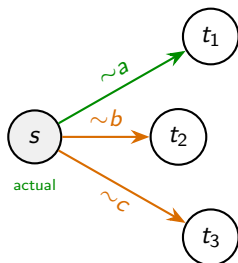


Desde s : solo una transición no-controlable $\sim a$.

- ▶ Elegir $\sim a \rightarrow t$ (forzosa)
 - STACK: sin cambios
 - IGNORE: sin cambios
 - Siguiente estado: t

El director *siempre* debe explorar no-controlables; no puede bloquearlas.

Caso 3 – Múltiples transiciones no-controlables



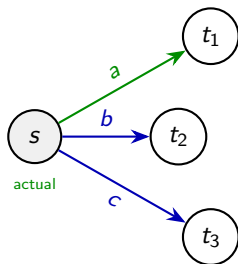
Desde s : tres no-controlables.

- ▶ Elegir $\sim a \rightarrow t_1$ (primera)
- ▶ Apilar $\sim b \rightarrow t_2$
- ▶ Apilar $\sim c \rightarrow t_3$
 - IGNORE: sin cambios
 - Siguiendo estado: t_1

Observación clave

Como s pertenece al director, *todas* sus no-controlables también pertenecen al director. No importa cuál se elige primero: el orden es irrelevante.

Caso 4 – Múltiples transiciones controlables



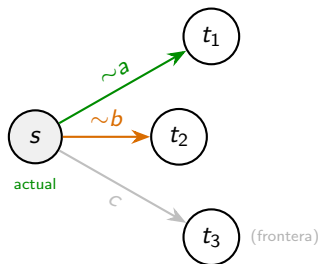
Desde s : tres controlables.

- ▶ Elegir $a \rightarrow t_1$ (*picker* de familia)
- ▶ Ignorar $b \rightarrow t_2$
- ▶ Ignorar $c \rightarrow t_3$
 - STACK: sin cambios
 - Siguiendo estado: t_1

Heurística de segundo nivel

O sea, tenemos otra heurística para decidir qué expandir (solo para controlables).

Caso 5a – Mezcla: picker elige no-controlable (null)

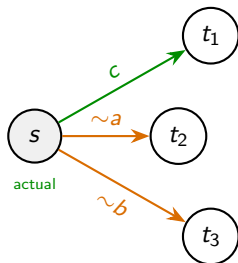


Picker devuelve `null`: no quiere elegir controlable.

- ▶ Elegir $\sim a \rightarrow t_1$
- ▶ Apilar $\sim b \rightarrow t_2$
- $c \rightarrow t_3$ queda en la *frontera*
- IGNORE: sin cambios

El director restringe todas las controlables; solo habilita las no-controlables.

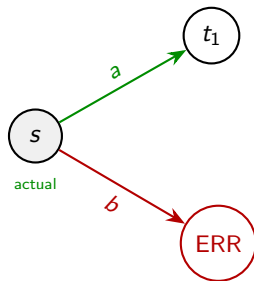
Caso 5b – Mezcla: picker elige controlable (non-null)



Picker devuelve c : elige la controlable.

- ▶ Elegir $c \rightarrow t_1$ (*picker* de familia)
- ▶ Apilar $\sim a \rightarrow t_2$
- ▶ Apilar $\sim b \rightarrow t_3$
 - IGNORE: otras controlables (si hubiera)

Las no-controlables van al STACK (no al IGNORE): deben explorarse todas.



Transiciones controlables a error

son descartadas antes de aplicar cualquier regla.

► Elegir $a \rightarrow t_1$

✗ $b \rightarrow \text{ERR}$ descartada

Un estado es *error* si está en el conjunto de errores del contexto, o si algún sub-estado del producto paralelo es ‘ERROR’.

Las no-controlables a error **no** se filtran (el director debe expandirlas).

Fallback – Pila e Ignorados

Cuando no hay transiciones desde el **estado actual** en la *frontera*:

1. **Pop stack** – tomar la transición del tope de la pila.
Pila → no-controlables diferidas de pasos anteriores.
2. **Pop ignore** – si la pila está vacía, tomar de la lista de ignorados.
Ignorados → controlables descartadas en decisiones anteriores.
3. **Primera de la frontera** – último recurso si ambas listas están vacías.

Invariante

Pila → ignorados garantiza que todas las ramas no-controlables se exploran *antes* de revisar las decisiones controlables diferidas.

Estados de la Planta

Estado de la planta: tupla de sub-estados, uno por cada LTS componente.

$$s = (s_0, s_1, \dots, s_{m-1})$$

Ejemplo:

Wait[2] | Wait[4] | Next[1] | Next[0] | off

Cada elemento es un sub-estado: Wait[2] es el sub-estado del primer LTS, Wait[4] el del segundo, etc.

Información valiosa

Los sub-estados son strings con semántica del componente. El picker los lee para decidir qué transición controlable expandir.

pickControllable por familia

pickControllable – DP (Dining Philosophers)

Candidatas: `take[i][j]` (filósofo i toma el fork j).

Prioridad:

- 1 **Ready:** elegir `take[i]` con menor i tal que el filósofo i está en ‘Ready’.
- 2 **Hungry:** elegir `take[i]` con menor i tal que el filósofo i está en ‘Hungry’ y el monitor *no* está en ‘Done’.
- 3 **Fallback:** primera transición.

Intuición

Ready significa que el filósofo ya obtuvo el primer fork y puede comer si toma el segundo. Atenderlo primero minimiza la espera. El índice mínimo da orden determinístico.

pickControllable – TL (Transfer Line)

Candidatas: $get[i]$ (recoger pieza en posición i).

Prioridad:

- 1 **Ya explorado:** elegir $get[i]$ cuyo estado destino ya fue visitado.
Cierra el ciclo de la línea de transferencia.
- 2 **Mayor índice:** elegir $get[i]$ con i máximo (que no vaya a error).
Avanza lo más posible hacia el final de la cadena.
- 3 **Fallback:** primera transición.

Intuición

La línea tiene un orden lineal natural. Cerrar el ciclo primero; si no es posible, avanzar al máximo i sin error.

pickControllable – AT (Air Traffic)

Candidatas: `approach[i]`, `descend[i][j]`.

Prioridad:

- 1 **Approach:** elegir `approach[i]` si las alturas del estado actual son *consecutivas* (sin huecos vacíos entre aviones).
- 2 **Descend:** elegir `descend[i][j]` con menor j tal que:
 - el avión i está en altura $j+1$ (justo encima del objetivo), y
 - la altura j está vacía (‘‘Empty’’) en el estado actual.
- 3 **Fallback:** primera transición.

Intuición

Primero completar aproximaciones cuando los aviones están ordenados; luego bajar el avión que está justo encima del primer hueco libre.

pickControllable – CM (Cat and Mouse)

Candidatas: `mouse[i] [move[j]]` (ratón i se mueve a posición j).

Regla:

$$\text{safePlace} = \left\lfloor \frac{2k + 1}{2} \right\rfloor$$

- Elegir el movimiento que minimiza $|j - \text{safePlace}|$ (destino más cercano al centro).
- Empate: menor índice de ratón i .
- **Fallback:** primera transición.

Intuición

El ratón debe alcanzar el centro del tablero (posición segura / goal). Mover siempre hacia la posición central minimiza los pasos hasta el objetivo.

pickControllable – BW (Bidding Workflow)

Candidatas: approve, refuse, assign[i].

Prioridad:

- ① **Approve:** elegir approve si no lleva a error.
- ② **Refuse:** elegir refuse si el primer sub-estado del estado *origen* es ‘‘Rejected’’.
- ③ **Assign:** elegir assign[i] con menor *i* tal que:
 - el crew *i* está en ‘‘Pending’’ o ‘‘Rejected[x]’’ en el estado actual, y
 - no lleva a error.
- ④ **Fallback:** primera transición.

Intuición

Aprobar y rechazar correctamente antes de asignar; dentro de las asignaciones, el orden por índice da determinismo.

pickControllable – TA (Travel Agency)

Candidatas: `purchase[i]`, `cancel[i]`, `agency.succ`, `agency.fail`.

Layout: `[Agency | AgencyMonitor | Svc(0) | SvcMon(0) | ... |`

`Svc(n-1) | SvcMon(n-1) | fluent]`

Prioridad:

- 1 **All Success:** si todos los sub-estados `SvcMon(i)` son ‘‘Success’’ y `agency.succ` no lleva a error $\rightarrow$ expandir `agency.succ`.
- 2 **agency.fail:** si `agency.fail` no lleva a error $\rightarrow$ expandirla.
- 3 **purchase[i]:** elegir `purchase[i]` con i más cercano al índice j del sub-estado `AgencyMonitor = ‘‘Disallow[j]’’`, tal que `SvcMon(i)  $\neq$  ‘‘Success’’`.
- 4 **Fallback:** primera transición.

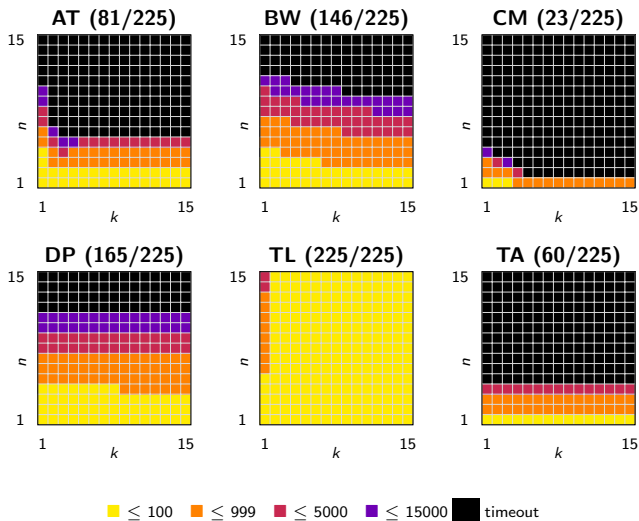
Intuición

Confirmar la compra cuando todos los servicios aceptaron; si no, comprar el servicio más cercano al índice actualmente denegado por el monitor de la agencia.

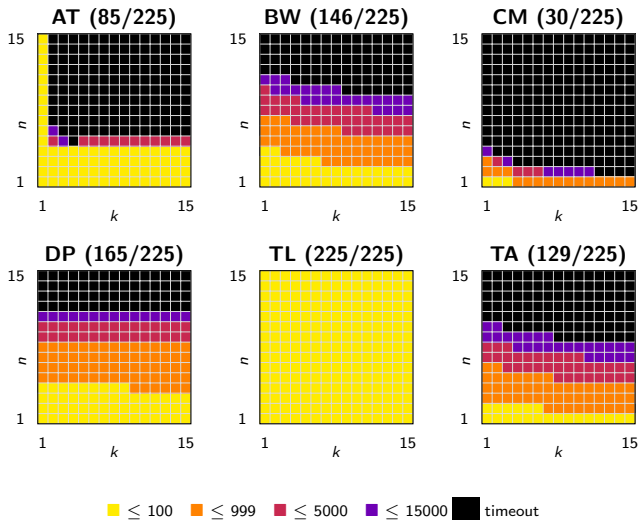
Resultados comparativos

Budget: 15 000 expansiones por instancia

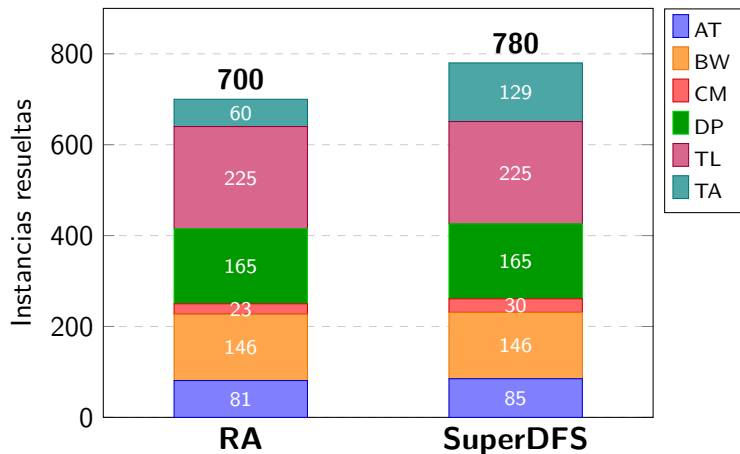
RA – Instancias resueltas (budget = 15 000)



SuperDFS – Instancias resueltas (budget = 15 000)



Comparativa – Instancias resueltas (budget = 15 000)



Super RL

Objetivo

Reemplazar el picker manual de SuperDFS por una red neuronal entrenada por RL.

Arquitectura:

- Red MLP: recibe el **vector de features** de cada transición candidata y devuelve un score.
- En cada paso se elige la transición con mayor score (*argmax*).
- Una red por familia (AT, DP, BW, CM, TL, TA).

Mismo esqueleto que SuperDFS

El algoritmo de exploración (pila, ignorados, fallback) es idéntico. Solo cambia el picker: en lugar de reglas codificadas a mano, usa la red.

Features: SUPER_CUSTOM

Familia	#	Features
AT	5	is_approach, target_empty, consecutive, height_norm, height_matches_slot
DP	5	is_take, ready, hungry, min_ready_idx, min_hungry
BW	7	is_approve, is_refuse, is_assign, doc_rejected, crew_pending, crew_rejected, min_eligible
CM	3	dist_to_safe_norm, is_min_dist, is_min_mouse
TL	2	dest_explored, is_max_idx
TA	6	is_succ, is_fail, is_purchase, all_success, this_success, dist_to_disallow_norm

Objetivo: aprender a imitar el mejor camino encontrado.

- ➊ **Ronda 0 – exploración aleatoria:** correr N episodios aleatorios. Guardar el episodio con mayor reward (= menos expansiones). **Ese es el mejor path.**
- ➋ **Rondas $k > 0$ – guided:** correr N episodios con la red actual (ϵ -greedy). Si alguno supera el mejor path global, actualizarlo.
- ➌ **Entrenamiento por ronda:** construir un dataset desde el mejor path y re-entrenar la red desde cero hasta convergencia.

Entrenamiento: Ranking de Feature Vectors

¿Cómo construir el dataset desde el mejor path?

Para cada feature vector (FV) único observado, se rastrean:

- **gano:** pasos donde ese FV fue *elegido*
- **perdio:** pasos donde ese FV estaba pero *no fue elegido*

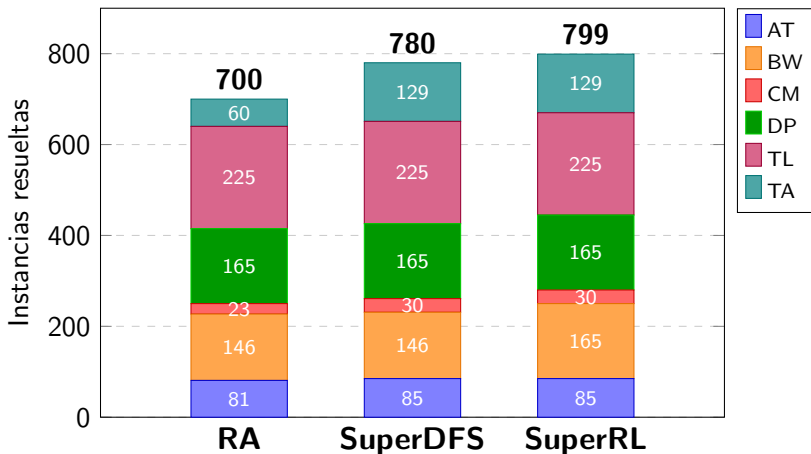
Dominancia: A domina a B si A ganó en algún paso donde B estaba y perdió.

Target:

$$y = \begin{cases} -1 & \text{si } \text{gano} = \emptyset \\ |\text{gano}| - |\{B : B \text{ domina a } A\}| & \text{si no} \end{cases}$$

Los targets se colapsan a rangos densos. La red se entrena con MSELoss; en inferencia se usa *argmax*.

Comparativa – RA vs SuperDFS vs SuperRL (budget = 15 000)



SuperRL – Instancias resueltas (budget = 15 000)

